

COMPUTER SCIENCE

*Doing webpages and making apps is not Computer Science,
but a consequence and discipline in the very broad field of Computation*

A collection of private note fragments, gathered and completed from GreyFox's Notebook

Shared for Jibaro Archives, marked by IIX.

2026 // Open Classification

ABSTRACT

The Architect's and Scientist's classification dilemma in a very modern field. Everybody wants to decide what's Computer Science without first observing its effects from the metal, to the kernel, all the way to operating systems and finally robots with a "brain" that interact with our physical environment.

Computer Science was born as formal mathematics, the study of computation as an abstract, logical discipline. It was never intended to be a natural science. Yet through a chain of events no single theorist planned, CS became something unprecedented: a universal substrate capable of instantiating any abstraction and then reaching through physical hardware into the real world. This paper traces that crossing, from Turing's theoretical machines to neural networks modeled on biological brains to embodied AI systems that act on physical reality, and argues that CS now represents a historically novel category of knowledge that is neither science, nor mathematics, nor engineering, but all three simultaneously, made possible by the unique property that its abstractions self-execute.

1 The Birth in Pure Abstraction

In 1936, Alan Turing published "On Computable Numbers" and did something that had never quite been done before. He did not conduct an experiment. He did not observe a natural phenomenon. He defined a machine that existed only on paper and asked: what can this machine, in principle, compute? That is a mathematical question, not a scientific one. [1]

Alongside Turing, Alonzo Church developed lambda calculus. John Von Neumann formalized stored-program architecture. These were not engineers building things. They were scientists and mathematicians drawing boundaries around a new formal object: computation itself. The computers that followed were almost incidental, physical embodiments of an idea that existed prior to them in pure logic. [2, 3]

"Computer science is no more about computers than astronomy is about telescopes."

– attr. Edsger W. Dijkstra

This is the origin point. CS did not begin as science. It began as the formal study of what is and is not computable, a branch of mathematical logic with scientific implications. The

name “Computer Science” was an institutional accident, a label applied when universities needed to house a new department that fit neither Mathematics nor Engineering cleanly.

“I wouldn’t be a computer scientist, because for engineers, you just make things work and implement them. But CS folk are more abstract, more abstract mathematicians. Lots of algorithms, and they make mental connections others can’t. They work with their minds a lot more. It’s a very mental thing, not just memorization.”

– GreyFox’s Electrical Engineering Professor

2 The Property That Changed Everything

Most formal disciplines stay abstract. A theorem in topology does not run. A proof in number theory does not consume electricity. The formal object and the physical world remain separate. CS broke this contract.

The Turing machine, a purely theoretical construct, has a direct physical realization: the transistor, the chip, the processor. When CS produces a formal system, that system can be executed. The abstraction does not merely describe a process. It becomes the process.

This is not a trivial distinction. It means that when CS advances, the advancement does not merely expand what we know. It expands what the world does. Every new algorithm is a new capability that can be instantiated in hardware and pointed at reality. No other formal discipline has this property at scale.

“They also use computers to do and solve math, and in multipolar fashion they use math to do and solve computers. It’s a very crazy thing from my perspective.”

– GreyFox’s Electrical Engineering Professor

3 Borrowing a Brain

The story of artificial intelligence is the story of CS reaching outside itself for the first time. Early AI was purely symbolic: logic rules, search trees, expert systems. It was still math dressed up as intelligence.

Then in the 1940s, Warren McCulloch and Walter Pitts published a model of how neurons fire. Frank Rosenblatt built the Perceptron in 1958, a computing device explicitly modeled on biological neural architecture. CS was no longer working from first principles alone. It was borrowing a design from nature, from neuroscience, from the wet machinery of animal cognition. [4, 5]

This matters. When a discipline imports a model from empirical science, it inherits the empirical assumptions of that model. Neural networks are not proven correct through formal proof. They are validated through testing against data from the real world. That is a scientific method, not a mathematical one.

4 Getting a Body

A neural network running in a server farm is still mostly abstract, its effects informational. But mount that network in a robot with cameras, force sensors, and actuators, and the abstraction now has limbs. It can pick up an object. It can navigate a room. It can fail because gravity exists and the floor is wet.

At this point, the system must be tested the way any physical system is tested, empirically, against the resistance of the real world. You cannot prove a priori that a robotic grasping algorithm will work on unfamiliar objects. You have to try it and observe. The world pushes back. That is the defining feature of science. [6]

5 The Classification Debate: A Primary Source

The following exchange occurred in a university classroom between the author and a professor of Electrical Engineering. It illustrates the demarcation problem from the inside of academia itself.

GreyFox:

“You say CompSci is closer to mathematicians than engineers. How can you say that when I know Electrical Engineers take physics, electromagnetism, and things like that?”

Professor of Electrical Engineering:

“Because my math is more applied, then I tighten screws, that’s it. Your math is a mental game and keeps going. So I would never do it. You work with the mind, and you must do mental connections others can’t. CompSci math is less daily life and more unknown abstract. Yes we do take calc and crazy math, but it’s more applied. It’s not as a mental screw.”

6 What Category Is CS Now?

CS therefore occupies all three of the major categories of rigorous knowledge simultaneously. It is mathematics when it produces formal proofs about algorithms and computability. It is engineering when it builds systems that must function under real-world physical constraints. It is empirical science when it trains models on data and validates them against observable reality.

This is not a weakness or an identity crisis. It is what makes CS unusual among formal disciplines. It may be the only formal discipline whose objects of study can be simultaneously proven, built, and observed acting on the world within the same research program. That triple nature is the source of its extraordinary reach.

The clearest demonstration is embodied robotics. A robot equipped with a neural network can navigate an unstructured environment, manipulate physical objects, and adapt its behavior based on sensor feedback. These systems must be tested empirically, in the real world, against physical constraints no simulation fully captures. The abstraction has a body, and the body has consequences.

7 Conclusion: The Universal Substrate

Computer Science began as mathematics asking what computation is. It became engineering when it built machines. It became science when those machines were trained on empirical data from the real world and tested against physical reality.

CS is a universal substrate, a medium that can run any abstraction and then project it into the physical world through hardware. Prior formal disciplines, mathematics, logic, linguistics, did not possess this execution property at scale. The chain runs: logic to circuit

to code to model to body to action. That chain became practically realizable with Turing's 1936 paper.

"We talked about the inevitable. . . uploaded human brain maps. The best computer is the human brain."

"Where would memories go? It becomes a philosophical problem."

– GreyFox's Electrical Engineering Professor

Turing asked what machines could think. Nobody asked what they would eventually touch. That was the more important question. We are living inside its answer.

References

References

- [1] A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, vol. 42, pp. 230–265, 1936. [doi:10.1112/plms/s2-42.1.230](https://doi.org/10.1112/plms/s2-42.1.230)
- [2] A. Church, "An Unsolvable Problem of Elementary Number Theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936. [doi:10.2307/2371045](https://doi.org/10.2307/2371045)
- [3] J. von Neumann, "First Draft of a Report on the EDVAC," Moore School of Electrical Engineering, University of Pennsylvania, Technical Report, 1945. [[archive.org](https://www.archive.org)]
- [4] W. S. McCulloch and W. Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943. [doi:10.1007/BF02478259](https://doi.org/10.1007/BF02478259)
- [5] F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. [doi:10.1037/h0042519](https://doi.org/10.1037/h0042519)
- [6] K. R. Popper, *The Logic of Scientific Discovery*. London: Hutchinson, 1959.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015. [doi:10.1038/nature14539](https://doi.org/10.1038/nature14539)
- [8] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

Note from Fox: *The best computer is the human brain at virtually no cost of processing power.*

GreyFox
Jibaro Archives, 2026

And so we talk. Words never spoken before. Or maybe already heard. Until and after class ends.